❸ Test function for initialize()

❹ Test function for decrement()

❺ Test function for increment()

❻ Tests for a modifier that controls the value of the counter to be >=0

Before running each test function, the counter is set to 100 by the function `beforeEach()`. For the tests, each `it()` `test` performs an operation, awaits the results, and asserts (checks) whether the results match the expected outcome. The `it()` for decrement shows these aspects. Observe the meaningful string parameter that describes the test: `'Success on decrement.'` It decrements by a value of 5 the counter value that has been set to `100` by the `beforeEach()` function. Then it awaits completion of this operation and checks if it is 95 using the `assert.equal(...)`. Use this syntax as a pattern for writing `it()` test scripts:

```
it('Success on decrement of counter.', async function () {
    await counter.decrement(5)
    assert.equal(await counter.get(), 95)
  })
```

Observe the syntax of the `it()` test. It has a description string for the test, and the function to be executed. Because the function is an `async()` function (asynchronous), the next line is the code to await completion of the decrement function, followed by the `assert` statement that compares the result to check whether it is correct. That’s it. Now you’re all set to write test scripts with simple independent tests defined by `it()`, using the test script in listing 10.2 as a guide.